

Sistema de Preguntas y Respuestas mediante Web Scraping y Modelos de Lenguaje de Gran Escala: Aplicación a la Fundación Valle del Lili

Frank Edward Daza Andrés Fernando López Viviana Valle Fernández Álvaro Julián Barco
Técnicas Avanzadas de IA Aplicadas en Modelos de Lenguaje
Universidad Autónoma de Occidente, Cali, Colombia
Docente: Jan Polanco Velasco

Resumen—Este artículo presenta el diseño e implementación de un sistema de preguntas y respuestas (Q&A) basado en información obtenida mediante *web scraping* aplicado al dominio institucional de la Fundación Valle del Lili. En su fase 2, el sistema se organiza en siete módulos secuenciales: (1) extracción automatizada mediante un *crawler* de búsqueda en anchura (BFS) que respeta las directivas de `robots.txt`, con un corpus de 1000 páginas HTML descargadas en aproximadamente 51 minutos; (2) conversión a Markdown con *front matter* YAML, produciendo 1000 documentos en `data/markdown/vallledellili-org/`; (3) recuperación léxica con el algoritmo BM25 Okapi, que selecciona y concatena los tres documentos de mayor puntuación como contexto enriquecido; (4) composición del *prompt* con instrucciones anti-alucinación; (5) generación de respuestas con ocho modelos de lenguaje disponibles bajo una interfaz uniforme —tres modelos locales en Ollama (llama3.1:8b, gemma4:e2b y gemma4:e4b) y cinco modelos en la nube por la API de OpenAI (gpt-4o, gpt-4o-mini, gpt-4-turbo, chatgpt-4o-latest y gpt-3.5-turbo)—; (6) un backend HTTP construido con FastAPI, Uvicorn y *sse-starlette* que expone el *pipeline* como servicio REST con transmisión *Server-Sent Events* (SSE) y un modo dual que consulta Ollama y OpenAI con una sola pasada BM25; y (7) un *frontend single page application* (SPA) desarrollado en React 19 + Vite 7 + TypeScript, con Tailwind CSS v4 y *shadcn/ui*, que renderiza la respuesta en tiempo real y permite editar el *prompt* de sistema sin modificar el código fuente. Se diseñó un protocolo de evaluación reproducible con 23 preguntas de referencia distribuidas en cinco categorías temáticas, con métricas de precisión de recuperación, Recall@3, tasa de abstención y latencia media por modelo.

Index Terms—recuperación aumentada por generación, BM25, *web scraping*, modelos de lenguaje grandes, FastAPI, *Server-Sent Events*, React, Vite, Ollama, OpenAI, procesamiento de lenguaje natural, inteligencia artificial

I. INTRODUCCIÓN

La creciente disponibilidad de modelos de lenguaje de gran escala (*Large Language Models*, LLM) ha abierto nuevas posibilidades para la construcción de sistemas de preguntas y respuestas capaces de interactuar en lenguaje natural sobre dominios específicos. Sin embargo, el uso de LLM sin restricción sobre el dominio de conocimiento conlleva el fenómeno de alucinación: la generación de información plausible pero factualmente incorrecta [1].

La Fundación Valle del Lili es una institución hospitalaria de referencia en Colombia que dispone de un sitio web institucional con información sobre servicios médicos, trámites

administrativos, especialidades y datos de contacto. Un sistema automatizado de Q&A sobre esta información reduciría la carga en canales de atención al cliente y mejoraría la accesibilidad para pacientes y público en general.

Este trabajo propone un *pipeline* RAG completo que aborda el problema en fases secuenciales bien delimitadas: adquisición de datos, procesamiento documental, recuperación de contexto relevante, generación de respuestas, exposición del *pipeline* como servicio HTTP y consumo desde una interfaz web moderna. La recuperación se implementa mediante el algoritmo BM25 Okapi [2], que selecciona los tres archivos más relevantes ante cada consulta y los concatena como contexto único. Estos documentos se presentan al LLM junto con un *prompt* de sistema que restringe la respuesta al dominio del corpus.

La contribución principal de este trabajo consiste en demostrar que es posible construir un sistema RAG funcional, trazable y libre de alucinaciones sin recurrir a bases de datos vectoriales ni a modelos de *embedding*, lo que simplifica el despliegue y reduce los requisitos computacionales en la fase inicial del proyecto. Como contribución adicional de la fase 2, el sistema se desacopla en un *backend* HTTP con FastAPI + SSE y un *frontend* SPA construido con React 19 + Vite 7 + Tailwind CSS v4 + *shadcn/ui*, lo que habilita interfaces de alto impacto visual, modo dual de comparación entre modelos locales y en la nube, y reutilización del API por scripts de evaluación y por los módulos posteriores del curso.

El artículo se organiza de la siguiente manera: la Sección II describe la arquitectura general; las Secciones III a VII detallan cada módulo; la Sección VIII presenta el protocolo de evaluación y los resultados; finalmente, las Secciones IX y X ofrecen la discusión y las conclusiones.

II. ARQUITECTURA DEL SISTEMA

El sistema sigue una arquitectura de tubería (*pipeline*) secuencial dividida en siete módulos funcionales, tal como se ilustra en la Figura 1. Cada módulo produce artefactos persistentes que sirven como entrada al módulo siguiente, lo que facilita la rejecución parcial del *pipeline* ante cambios en el corpus o en los parámetros del sistema.

La Tabla I resume las tecnologías empleadas en cada módulo y los artefactos que produce.

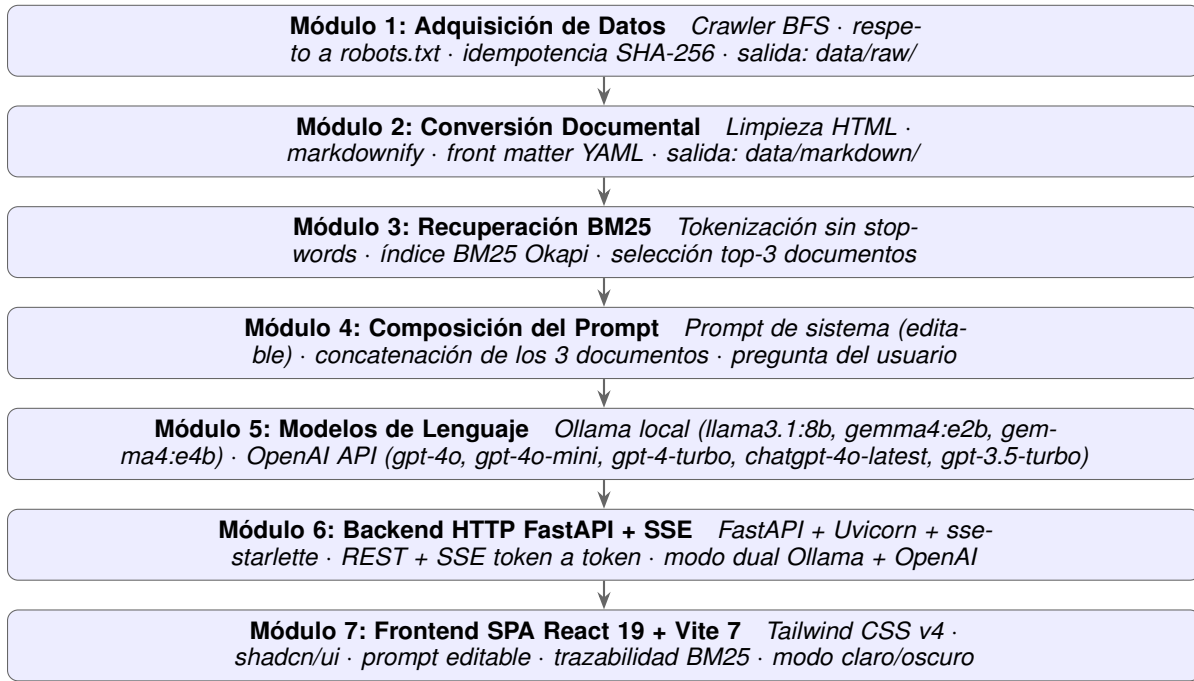


Figura 1. Arquitectura general del *pipeline* RAG en la fase 2. El *pipeline* de datos (módulos 1–5) se expone como servicio HTTP (módulo 6) y se consume desde una *single page application* desacoplada (módulo 7).

Cuadro I
STACK TECNOLÓGICO POR MÓDULO

Módulo	Tecnología	Artefacto de salida
Adquisición	requests, BeautifulSoup4	HTML + JSON sidecar
Conversión	markdownify, PyYAML	Markdown + front matter
Recuperación	rank-bm25	Top-3 documentos
Prompt	composición propia	Lista de mensajes
LLM	Ollama, OpenAI SDK	Texto de respuesta
Backend	FastAPI, Uvicorn, sse-starlette, pydantic-settings	API REST + eventos SSE
Frontend	React 19, Vite 7, TypeScript, Tailwind v4, shadcn/ui, Vercel AI SDK	SPA web con streaming

Cuadro II
PARÁMETROS DE CONFIGURACIÓN DEL *crawler* (FASE 2)

Parámetro	Valor utilizado	Descripción
url_inicio	valledellili.org	URL semilla
max_paginas	1 000	Páginas máximas a visitar
profundidad_max	8	Niveles de enlace
delay_segundos	1.5s	Pausa entre peticiones
timeout	30s	Tiempo máximo HTTP
reintentos	3	Reintentos ante error

Nota: La interfaz Gradio del MVP fase 1 se conserva como código legacy en `src/app/legacy/app_gradio.py` [5]; la experiencia oficial del usuario es ahora la SPA del módulo 7.

III. MÓDULO DE ADQUISICIÓN DE DATOS

III-A. Diseño del Crawler

El módulo de *scraping* implementa un *crawler* BFS sobre el dominio <https://valledellili.org/>. La clase `Crawler` acepta una configuración con los parámetros descritos en la Tabla II. Los valores corresponden a los efectivamente usados en la última ejecución del módulo en la fase 2.

III-B. Respeto a `robots.txt` e Idempotencia

Antes de visitar cualquier URL, el sistema consulta la clase `GestorRobots`, que carga y analiza el archivo `robots.txt` del dominio con carga perezosa (*lazy loading*). Si la directiva prohíbe el acceso o la lectura falla, el gestor adopta un

modo conservador que rechaza todas las rutas. El *Crawl-Delay* declarado en `robots.txt` se respeta si supera el `delay_segundos` configurado.

Para garantizar idempotencia, cada página descargada se persiste junto a un archivo *sidecar* JSON que almacena su hash SHA-256. En ejecuciones posteriores, si el contenido no ha cambiado, la página se omite sin realizar una nueva petición HTTP.

Ante errores transitorios (códigos 5xx o 429), el sistema aplica retroceso exponencial con esperas de 1, 2 y 4 segundos entre intentos sucesivos.

III-C. Salida del Módulo

Por cada página exitosa, el *crawler* genera en `data/raw/valledellili-org/` un par de archivos:

- **HTML:** contenido bruto de la respuesta HTTP.
- **JSON sidecar:** URL canónica, código de estado, hash SHA-256, profundidad de rastreo y marca de tiempo.

Las descargas se registran de forma acumulativa en `data/raw/_log.jsonl`, lo que permite auditar qué URLs

se visitaron, cuáles se omitieron por `robots.txt` o por hash idéntico, y cuáles fallaron en red.

III-D. Resultados de la ejecución

La última ejecución del módulo se invocó con el comando:

```
1 uv run python -m scripts.scrape \
2   --max-paginas 1000 \
3   --profundidad-maxima 8
```

Listing 1. Comando de ejecución del *crawler* en la fase 2

La Tabla III resume las métricas observadas y la Figura 2 muestra la salida resumida del *runner* de ejecución. Los 20 errores corresponden a URLs que fallaron en red o devolvieron un código HTTP distinto de 200; no impidieron completar el lote solicitado de 1000 descargas exitosas.

Cuadro III
RESULTADOS DEL *scraping* (CORRIDA FASE 2)

Métrica	Valor
Descargas exitosas	1000
Omitidas por <code>robots.txt</code>	0
Omitidas por hash (idempotencia)	0
Errores (red o HTTP $\neq$ 200)	20
Duración total	$\approx$ 51 min

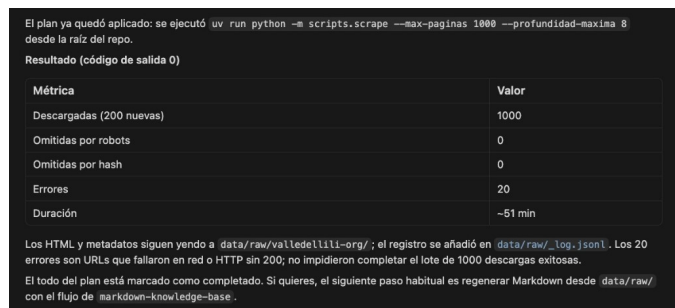


Figura 2. Salida del *runner* `scripts.scrape` tras la ejecución con `-max-paginas 1000 -profundidad-maxima 8`: 1000 descargas exitosas, 0 omitidas por `robots.txt` ni por hash, 20 errores y una duración aproximada de 51 min.

IV. MÓDULO DE PROCESAMIENTO DOCUMENTAL

IV-A. Limpieza del HTML

Antes de la conversión, se elimina el ruido léxico mediante BeautifulSoup4: se suprimen los elementos `<script>`, `<style>`, `<nav>`, `<header>`, `<footer>`, `<iframe>` y los banners de cookies. Esta etapa mejora directamente la calidad de la recuperación BM25 al reducir términos irrelevantes en el vocabulario del corpus.

IV-B. Conversión a Markdown

La conversión HTML $\rightarrow$ Markdown se realiza con la biblioteca `markdownify` [6], configurada con encabezados ATX (#) y viñetas con guion. El resultado se normaliza para eliminar saltos de línea consecutivos excesivos (máximo dos seguidos).

IV-C. Front Matter YAML

Cada archivo Markdown incorpora un bloque de metadatos YAML al inicio:

```
1 ---
2 source_url: https://valledellili.org/atencion-al-
3   paciente/
4 titulo: "Atención al Paciente"
5 seccion: atencion-al-paciente
6 fecha_extraccion: '2026-04-27'
7 idioma: es
8 hash: a3f2c1d9...
```

Listing 2. Ejemplo de front matter generado automáticamente

El campo `source_url` es fundamental para la trazabilidad: se expone al usuario junto a cada respuesta generada. El campo `hash` permite la idempotencia de re-exportaciones.

IV-D. Salida del Módulo

Los archivos Markdown se almacenan en `data/markdown/valledellili-org/`. El script `export_markdown.py` es idempotente: omite la re-conversión si el hash del HTML coincide con el registrado en el Markdown existente, a menos que se invoque con la bandera `-forzar`.

IV-E. Resultados de la exportación a Markdown

Tras la corrida de *scraping* descrita en la Sección III-D, el comando:

```
1 uv run python -m scripts.export_markdown
```

Listing 3. Comando de exportación del corpus crudo a Markdown

produjo el corpus textual definitivo utilizado por el recuperador BM25. La Tabla IV resume las métricas observadas y la Figura 3 muestra la salida del script. El corpus pasa así de los 45 documentos aproximados del MVP fase 1 a 1000 documentos `.md` en `data/markdown/valledellili-org/`, cada uno con su bloque de *front matter* YAML y su hash SHA-256.

Cuadro IV
RESULTADOS DE LA EXPORTACIÓN A MARKDOWN (CORRIDA FASE 2)

Métrica	Valor
Archivos <code>.md</code> creados	1000
Archivos actualizados	0
Omitidos por hash	0
Errores de conversión	0
Duración total	$\approx$ 32 s

V. MÓDULO DE RECUPERACIÓN BM25

V-A. Algoritmo BM25 Okapi

La recuperación de información se basa en el algoritmo BM25 Okapi [2], una función de puntuación probabilística que pondera la frecuencia de los términos de la consulta en cada documento y normaliza por la longitud del documento. La puntuación de un documento D ante una consulta $Q = \{q_1, q_2, \dots, q_n\}$ se define como:

```

Listo. Se ejecutó desde la raíz del repo:

uv run python -m scripts.export_markdown

Salida: código de salida 0, -32 s.

Resumen:
• creados: 1000
• actualizados: 0
• omitidos_por_hash: 0
• errores: 0

Los .md quedaron en data/markdown/valledellili-org/. El todo del plan está marcado como completado. Si más adelante
quieres regenerar todo aunque el hash coincida, puedes usar uv run python -m scripts.export_markdown --forzar.

```

Figura 3. Salida del script `scripts.export_markdown`: 1000 archivos `.md` creados, 0 actualizados, 0 omitidos por hash y 0 errores en una duración aproximada de 32 s.

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) (k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + b \frac{|D|}{\overline{dl}}\right)} \quad (1)$$

donde:

- $f(q_i, D)$: frecuencia del término q_i en el documento D .
- $|D|$: longitud del documento en número de tokens.
- $\overline{dl}$: longitud media de los documentos del corpus.
- $k_1 = 1.5$: parámetro de saturación de frecuencia de término.
- $b = 0.75$: parámetro de normalización de longitud.
- $\text{IDF}(q_i)$: frecuencia inversa de documento:

$$\text{IDF}(q_i) = \log \frac{N - \text{df}(q_i) + 0.5}{\text{df}(q_i) + 0.5} \quad (2)$$

siendo N el total de documentos del corpus y $\text{df}(q_i)$ el número de documentos que contienen el término q_i .

V-B. Tokenización

Antes de indexar el corpus y de procesar cada consulta, los textos se someten al siguiente proceso:

1. **Normalización Unicode**: descomposición NFD y eliminación de marcas combinantes (tildes, diéresis), de modo que, por ejemplo, *médico* se tokeniza igual que *medico*.
2. **Minúsculas**: conversión completa a minúsculas.
3. **Extracción de tokens**: expresión regular `\w+` sobre texto Unicode.
4. **Filtrado**: se descartan tokens de longitud < 2 y los pertenecientes a la lista de *stopwords* del español (*de, la, el, en, y, que, los, las, del, un, una, por, con, para, se, su, es, al, lo, a, o*, entre otras).

V-C. Selección de Top-3 Documentos

A diferencia de los enfoques que recuperan un único documento, este sistema selecciona los **tres documentos con mayor puntuación BM25**. Formalmente, dado el conjunto de puntuaciones $\{s_1, s_2, \dots, s_M\}$ calculadas para los M documentos del corpus, se selecciona el subconjunto:

$$\mathcal{C} = \arg \max_{|\mathcal{C}|=3, D \in \mathcal{D}} \text{BM25}(D, Q), \quad s_j > 0 \quad (3)$$

Los documentos en $\mathcal{C}$ se ordenan de mayor a menor puntuación y se concatenan para conformar el contexto que se envía al LLM. Esta estrategia reduce el riesgo de que un único documento no contenga toda la información necesaria para responder preguntas que abarcan varias secciones del sitio.

V-D. Implementación

El módulo `src/retrieval/recuperador.py` implementa la clase `RecuperadorBm25` usando la biblioteca `rank-bm25` [3]. El índice se construye en memoria al inicializar la clase, cargando todos los archivos `*.md` del directorio `data/markdown/valledellili-org/`. Sobre el corpus actual de 1000 documentos Markdown, la indexación completa se realiza en el arranque del servidor FastAPI en unos pocos segundos, lo que permite reindexar bajo demanda a través del endpoint `POST /api/recargar-corpus` sin reiniciar el servicio. El método `buscar(pregunta)` retorna una lista de objetos `DocumentoRecuperado` con los atributos: `ruta`, `titulo`, `source_url`, `contenido` y `score`.

VI. INTEGRACIÓN CON MODELOS DE LENGUAJE

VI-A. Composición del Prompt

Una vez recuperados los tres documentos $\mathcal{C} = \{D_1, D_2, D_3\}$, el módulo `src/qa/prompt.py` construye la lista de mensajes que se envía al LLM mediante la función `componer_mensajes()`. El mensaje de sistema incluye el *prompt* de sistema y el contexto formado por la concatenación de los tres documentos, separados por delimitadores explícitos con su puntuación BM25:

```

1 [
2   {
3     "role": "system",
4     "content": (
5       "<PROMPT_SISTEMA>\n\n"
6       "=== DOCUMENTO 1 (score: 18.4) ===\n"
7       "<contenido D1>\n\n"
8       "=== DOCUMENTO 2 (score: 12.1) ===\n"
9       "<contenido D2>\n\n"
10      "=== DOCUMENTO 3 (score: 8.7) ===\n"
11      "<contenido D3>\n\n"
12      "Responde usando SOLO el contexto anterior."
13    )
14  },
15  {
16    "role": "user",
17    "content": "<PREGUNTA_DEL_USUARIO>"
18  }
19 ]

```

Listing 4. Estructura del mensaje enviado al LLM

VI-B. Prompt de Sistema Anti-Alucinación

El *prompt* de sistema predeterminado instruye al LLM para que actúe como "Lili", asistente virtual de la Fundación Valle del Lili. Las directivas fundamentales son:

- Responder **únicamente** con información presente en el contexto entregado.
- Emitir la frase *"No tengo información suficiente"* si la consulta no puede responderse con el contexto disponible.

- No inventar números de teléfono, correos electrónicos, direcciones ni especialidades médicas.
- Mantener un tono profesional y cálido en español colombiano.
- Limitar la respuesta a seis oraciones (salvo listas o pasos) y estructurarla en Markdown.

Estas restricciones constituyen la estrategia primaria de mitigación de alucinaciones, complementada por la restricción de dominio que impone el RAG.

VI-C. Modelos Soportados

La Tabla V describe los ocho modelos integrados en el sistema, accesibles desde la interfaz sin modificar el código. Las listas se mantienen en `src/qa/cliente_ollama.py` (MODELOS_OLLAMA_SOPORTADOS) y `src/qa/cliente_openai.py` (MODELOS_OPENAI_SOPORTADOS); el endpoint `GET /api/modelos` las expone al *frontend* para poblar el selector de modelos.

Cuadro V
MODELOS DE LENGUAJE INTEGRADOS EN EL SISTEMA

Modelo	Backend	Acceso
llama3.1:8b	Ollama	Local
gemma4:e2b	Ollama	Local
gemma4:e4b	Ollama	Local
gpt-4o	OpenAI	API / nube
gpt-4o-mini	OpenAI	API / nube
gpt-4-turbo	OpenAI	API / nube
chatgpt-4o-latest	OpenAI	API / nube
gpt-3.5-turbo	OpenAI	API / nube

VI-C1. Modelos Locales con Ollama: llama3.1:8b [7] y las variantes gemma4:e2b y gemma4:e4b [8] se ejecutan localmente mediante Ollama [4], que expone una API REST en `http://localhost:11434`. El cliente `ClienteOllama` realiza peticiones `POST /api/chat` con `temperature = 0.2` y una ventana de contexto configurable por la variable `OLLAMA_NUM_CTX` (acotada al rango [1 024, 16 384], con valor por defecto de 8 192 tokens). Ante errores de conexión (`OllamaNoAccesibleError`) o modelo no disponible (`ModeloNoDisponibleError`), el mensaje se propaga al *frontend* como evento SSE {"tipo": "error"} y se muestra al usuario en un *toast* con la causa probable y la acción correctiva.

VI-C2. Modelos en la Nube con OpenAI: El cliente `src/qa/cliente_openai.py` acepta cinco identificadores válidos para el parámetro `model` de `chat.completions:` `gpt-4o` [9], `gpt-4o-mini`, `gpt-4-turbo`, `chatgpt-4o-latest` y `gpt-3.5-turbo`. La selección se hace en tiempo de ejecución desde el *frontend* sin modificar el código. La clave de autenticación se configura con la variable de entorno `OPENAI_API_KEY` en el archivo `.env`, leída con `pydantic-settings`. El acceso se realiza mediante la API oficial de OpenAI [10].

Ante errores HTTP 429 (`RateLimitError`), el cliente aplica hasta tres reintentos con *backoff* que respeta el encabezado

`Retry-After` si la respuesta lo expone; en ausencia de dicho encabezado, usa una espera exponencial acotada. Otros errores de autorización o facturación (`AuthenticationError`, `PermissionDeniedError`) se traducen a mensajes explícitos en español que el *frontend* renderiza en un *toast* de error. El cliente implementa la misma interfaz que `ClienteOllama`, lo que permite al `PipelineQa` orquestrar ambos *backends* de forma uniforme, incluyendo el modo dual descrito en la Sección VII.

VII. BACKEND HTTP FASTAPI + SSE Y FRONTEND REACT 19

En la fase 2 del proyecto, la interfaz monolítica Gradio del MVP se reemplazó por una arquitectura cliente-servidor de dos capas: un *backend* HTTP en `src/api/` que expone el `PipelineQa` como servicio REST con *streaming* SSE, y un *frontend single page application* desarrollado en `frontend/`. Esta separación, discutida con mayor detalle en [19], permite reutilizar el API desde *scripts* de evaluación, cuadernos de experimentación y futuros módulos del curso sin acoplarse a la capa de presentación.

VII-A. Backend HTTP FastAPI

El *backend* se construye con FastAPI [11], Uvicorn [12] como servidor ASGI y `sse-starlette` [13] para los endpoints de *streaming*. La configuración se carga desde `.env` con `pydantic-settings`, incluyendo las variables `ALLOWED_ORIGINS` (dominios permitidos en `CORSMiddleware`) y `OPENAI_API_KEY`.

Al arrancar la aplicación, un *context manager* de ciclo de vida instancia una única vez el `PipelineQa`, lo que evita reconstruir el índice BM25 en cada petición. Las dependencias HTTP acceden al pipeline mediante `Depends (obtener_pipeline)`. Los mensajes de error se traducen a códigos HTTP estándar con cuerpos JSON tipados por modelos `Pydantic v2`. La Tabla VI resume los endpoints disponibles.

Cuadro VI
ENDPOINTS HTTP EXPUESTOS POR SRC/API/

Método	Ruta	Descripción
GET	/api/salud	Health check
GET	/api/modelos	Modelos Ollama y OpenAI disponibles
GET	/api/prompt-defecto	Prompt de sistema por defecto
POST	/api/recargar-corpus	Reindexa BM25 desde disco
POST	/api/qa	Respuesta sincrónica
POST	/api/qa/stream	Streaming SSE (un motor)
POST	/api/qa/dual/stream	Streaming SSE modo dual

VII-B. Streaming SSE Token a Token

Los endpoints de *streaming* devuelven una secuencia de eventos *Server-Sent Events* con el campo `data` serializado en JSON. Cada evento declara un campo `tipo` (`token`, `final`, `fuentes` o `error`) y, en los casos aplicables, un campo `motor` que identifica si proviene de Ollama o de OpenAI. El servidor emite además comentarios SSE : `keepalive` cada quince segundos para mantener viva la conexión a través de *proxies* intermediarios.

```

1 data: {"tipo": "token", "motor": "ollama", "texto":
  "La "}
2 data: {"tipo": "token", "motor": "ollama", "texto":
  "Fundacion "}
3 ...
4 data: {"tipo": "final", "motor": "ollama",
  "texto": "...", "latencia_ms": 1823}
5
6 data: {"tipo": "fuentes", "fuentes": [...]}
7 : keepalive

```

Listing 5. Ejemplo simplificado de flujo SSE emitido por POST /api/qa/stream

En el navegador, el objeto `EventSource` nativo no permite enviar peticiones POST con cuerpo JSON; por esta razón, el *frontend* implementa un cliente propio basado en `fetch` combinado con `ReadableStream` y `TextDecoderStream`. Este patrón mantiene la semántica SSE y a la vez soporta cuerpos de petición arbitrarios.

VII-C. Modo Dual Ollama + OpenAI

El modo dual, gatillado cuando el usuario activa simultáneamente ambos *backends*, se resuelve con una única invocación a `preparar_contexto_inferencia` (pregunta, prompt) que construye los mensajes una sola vez a partir del top-3 BM25. El endpoint POST /api/qa/dual/stream reutiliza ese contexto y emite primero todos los eventos con motor: "ollama" y, a continuación, todos los eventos con motor: "openai". Esto evita duplicar el costo de la recuperación BM25 y entrega al usuario dos respuestas independientes que comparten exactamente las mismas fuentes. El *frontend* renderiza ambas en el componente `DualResponseView`, con dos columnas independientes y una zona de fuentes compartida.

VII-D. Frontend React 19 + Vite 7

El *frontend* vive en la carpeta `frontend/` y se compone de React 19 [14] servido por Vite 7 [15] con TypeScript en modo estricto. La gestión del paquete usa `pnpm`. Los estilos se implementan con Tailwind CSS v4 [16] a través del plugin oficial para Vite. Los componentes de interfaz se construyen sobre `shadcn/ui` [17] (Radix UI + `lucide-react`), lo que facilita accesibilidad ARIA de forma nativa. La integración con el *streaming* SSE se apoya en Vercel AI SDK [18] con un *transport* personalizado que consume nuestro propio endpoint.

El sistema de diseño adopta tokens CSS alineados a la identidad institucional de la Fundación Valle del Lili: morado primario #6B21A8, teal complementario #0D9488 y un par de fondos claro/oscuro gestionados por `next-themes` con persistencia en `localStorage`. La tipografía usa Inter para cuerpo y JetBrains Mono para código. El *runtime* de datos emplea `@tanstack/react-query` v5 para cachear GET /api/modelos y GET /api/prompt-defecto, `react-markdown` con `remark-gfm` y `shiki` para render de Markdown enriquecido con resaltado de sintaxis, y `sonner` para notificaciones tipo *toast*. La Tabla VII resume las vistas principales.

VII-E. Prompt de Sistema Editable y Trazabilidad

La SPA conserva las dos funcionalidades diferenciales heredadas del MVP:

Cuadro VII
VISTAS REACT PRINCIPALES DEL *frontend*

Vista	Función
AppShell	Encabezado, barra lateral colapsable, área principal y conmutador de tema
Chat	Orquesta llamadas SSE y estado de la conversación
MessageBubble	Render Markdown con <code>shiki</code> y <i>auto-scroll</i>
DualResponseView	Dos columnas Ollama OpenAI con la misma pregunta
SourcesPanel	Cards con título, URL y <i>score</i> BM25
SettingsPanel	Toggles de motor, selectores de modelo, <i>sliders</i> y editor de <i>prompt</i> de sistema

- **Prompt de sistema editable:** el componente `SystemPromptEditor` permite modificar en caliente las instrucciones anti-alucinación sin reiniciar el servicio. Un botón restaura el *prompt* de fábrica consumido desde GET /api/prompt-defecto.
- **Trazabilidad BM25:** al recibir el evento {"tipo": "fuentes"}, la SPA renderiza en `SourcesPanel` los tres documentos recuperados con su título, su `source_url` clicable y su puntuación BM25. El evento {"tipo": "final"} añade la latencia total en milisegundos y el identificador del modelo empleado, lo que permite al usuario auditar de qué fuentes proviene la respuesta.

Mientras la respuesta está en curso, el botón de enviar se deshabilita y el área de mensaje muestra un *skeleton loader* que se reemplaza por el texto final al recibir el evento {"tipo": "final"} del motor correspondiente. En caso de error (por ejemplo, Ollama no accesible o cuota de OpenAI agotada) se emite un evento {"tipo": "error"} que se traduce en un *toast* explicativo con la causa probable y la acción sugerida.

VII-F. Despliegue y Comandos Canónicos

En desarrollo, el *backend* corre con `uv run uvicorn src.api.main:app --reload --port 8000` y el *frontend* con `pnpm --dir frontend dev` en el puerto 5173; Vite enruta todas las peticiones /api al puerto 8000 mediante su *proxy* de desarrollo. En producción, `pnpm --dir frontend build` genera `frontend/dist/`, que FastAPI sirve como `StaticFiles` desde el mismo proceso.

VIII. EVALUACIÓN

VIII-A. Protocolo de Evaluación

Se diseñó un banco de 23 preguntas de referencia almacenadas en `tests/qa/preguntas_evaluacion.yml`. Cada entrada contiene: identificador, texto de la pregunta, categoría temática y, cuando aplica, el nombre del archivo Markdown esperado como resultado de la recuperación. Las preguntas se distribuyeron en cinco categorías (Tabla VIII).

El script `scripts/evaluar_qa.py` se ejecuta contra el corpus actualizado de 1 000 documentos Markdown y sobre

Cuadro VIII
DISTRIBUCIÓN DEL BANCO DE PREGUNTAS POR CATEGORÍA

Categoría	Preguntas	Con archivo esperado
Institucional	8	8
Servicios	5	5
Procesos	7	7
Contacto	2	2
Fuera de alcance	1	—
Total	23	22

los ocho modelos descritos en la Tabla V (tres modelos Ollama locales y cinco modelos OpenAI en la nube). Por cada combinación pregunta-modelo genera un informe Markdown en `data/processed/evaluaciones/<modelo>__<fecha>.md`. Las métricas calculadas son:

- **Precisión de recuperación (P@1)**: proporción de preguntas donde el documento con mayor puntuación BM25 coincide con el archivo esperado.
- **Recall@3**: proporción de preguntas donde el documento esperado se encuentra entre los tres recuperados.
- **Tasa de abstención**: proporción de respuestas que contienen la frase "No tengo información suficiente".
- **Latencia media**: tiempo promedio de generación en milisegundos.

VIII-B. Resultados

La Tabla IX presenta los resultados obtenidos al ejecutar el protocolo de evaluación sobre los tres modelos.

Cuadro IX
RESULTADOS DE EVALUACIÓN POR MODELO

Modelo	P@1	Recall@3	Abstención	Latencia (ms)
llama3.1:8b	—	—	—	—
gemma4:e2b	—	—	—	—
gemma4:e4b	—	—	—	—
gpt-4o	—	—	—	—
gpt-4o-mini	—	—	—	—
gpt-4-turbo	—	—	—	—
chatgpt-4o-latest	—	—	—	—
gpt-3.5-turbo	—	—	—	—

Nota: Los valores de la Tabla IX serán completados una vez ejecutado el protocolo de evaluación sobre el corpus definitivo.

VIII-C. Análisis Cualitativo

[Esta sección presentará ejemplos representativos de respuestas generadas por cada modelo, incluyendo: (a) casos donde los tres modelos producen respuestas correctas y trazables, (b) casos donde el modelo emite la respuesta de abstención ante preguntas fuera del corpus, y (c) casos de divergencia entre modelos ante la misma consulta.]

IX. DISCUSIÓN

IX-A. Ventajas del Enfoque BM25 sin Embeddings

La decisión de emplear BM25 en lugar de *embeddings* densos y bases de datos vectoriales responde a criterios de

simplicidad, trazabilidad y eficiencia. BM25 no requiere un modelo de *embedding* preentrenado, no demanda GPU para la indexación y produce puntuaciones directamente interpretables en términos de frecuencia de términos. Sobre el corpus actual de 1 000 documentos Markdown el índice se reconstruye en unos pocos segundos al arranque del *backend*, y cada búsqueda se resuelve en tiempo $O(n \cdot |Q|)$ donde n es el número de documentos.

Además, el sistema de recuperación con top-3 documentos amplía la cobertura contextual respecto a la recuperación de documento único, permitiendo que el LLM sintetice información distribuida en varias páginas del sitio web.

IX-B. Ventajas de la Separación Backend / Frontend

La arquitectura FastAPI + SSE desacoplada del consumidor web introduce ventajas prácticas frente al monolito Gradio del MVP. El mismo API HTTP puede ser invocado por el *frontend* React, por los *scripts* de evaluación (`scripts/evaluar_qa.py`) o por futuros clientes —incluidos los experimentos del módulo 2 sobre *chunking* semántico y recuperación densa— sin duplicar la lógica de orquestación del `PipelineQa`. El *frontend*, al consumir un contrato JSON/SSE estable, evoluciona con ciclos de iteración propios (diseño, accesibilidad, branding) sin bloquearse en la capa Python. Finalmente, la interfaz uniforme de los clientes de Ollama y OpenAI facilita el modo dual: el costo de recuperación BM25 se paga una sola vez y el mismo contexto alimenta ambos motores en paralelo, lo que habilita comparativas directas entre modelos locales y en la nube.

IX-C. Limitaciones Identificadas

1. **Truncamiento de contexto**: si la concatenación de los tres documentos supera los 8 192 tokens del parámetro `num_ctx` por defecto, Ollama trunca el contexto de forma silenciosa, lo que puede degradar la calidad de la respuesta. El corpus ampliado a 1 000 documentos incrementa la probabilidad de que los archivos recuperados sean largos y exijan elevar `OLLAMA_NUM_CTX` o considerar *chunking*.
2. **Ambigüedad léxica**: BM25 no captura similitud semántica; términos sinónimos o expresiones parafraseadas pueden no coincidir léxicamente con el vocabulario del corpus, produciendo recuperaciones incorrectas.
3. **Dependencia del servicio Ollama**: los modelos locales requieren que el proceso Ollama esté en ejecución y que los modelos hayan sido descargados previamente.
4. **Costo de la API**: el uso de los modelos OpenAI —especialmente `gpt-4o` y `gpt-4-turbo`— genera consumo de tokens facturables; el cliente `src/qa/cliente_openai.py` implementa manejo de `RateLimitError` con *backoff*, pero el equipo debe vigilar los límites de cuota del plan contratado.
5. **Doble runtime de desarrollo**: la separación *backend* Python (`uv`) y *frontend* Node (`pnpm`) requiere instalar ambos entornos para desarrollar, lo que incrementa el costo de *onboarding* respecto al monolito Gradio.

IX-D. Trabajo Futuro

Para la siguiente fase del proyecto se contempla migrar a una arquitectura basada en *chunking* semántico, modelos de *embedding* y una base de datos vectorial (Chroma o FAISS), lo que permitirá superar las limitaciones léxicas de BM25 y manejar documentos de mayor longitud sin riesgo de truncamiento.

X. CONCLUSIONES

Este trabajo presentó un sistema RAG completo para la Fundación Valle del Lili, construido con herramientas de código abierto y ejecutable en hardware de consumo sin necesidad de GPU. Los aportes principales son:

1. Un *pipeline* de adquisición y procesamiento documental robusto que respeta `robots.txt` y garantiza idempotencia mediante hashes SHA-256, con un corpus escalado en la fase 2 a 1 000 páginas HTML y 1 000 documentos Markdown con *front matter* YAML.
2. Un mecanismo de recuperación BM25 que selecciona y concatena los tres documentos más relevantes, ampliando la cobertura contextual respecto a la recuperación de documento único.
3. La integración de ocho modelos de lenguaje heterogéneos—tres locales con Ollama (`llama3.1:8b`, `gemma4:e2b`, `gemma4:e4b`) y cinco en la nube con la API de OpenAI (`gpt-4o`, `gpt-4o-mini`, `gpt-4-turbo`, `chatgpt-4o-latest` y `gpt-3.5-turbo`)—bajo una interfaz uniforme que permite comparación directa sin modificar el código.
4. Una separación clara en dos capas: un *backend* HTTP FastAPI + `sse-starlette` que expone el `PipelineQa` como servicio REST con *streaming* SSE token a token y con modo dual Ollama + OpenAI sobre una sola pasada BM25.
5. Un *frontend single page application* construido con React 19 + Vite 7 + TypeScript + Tailwind CSS v4 + `shadcn/ui`, que encapsula una experiencia de usuario de alto impacto visual con identidad institucional, modo claro y oscuro, *prompt* de sistema editable en caliente y panel de trazabilidad BM25.
6. Un protocolo de evaluación reproducible con 23 preguntas de referencia y cuatro métricas de calidad de recuperación y generación, ejecutable contra cualquier subconjunto de los ocho modelos integrados.

Los resultados de la evaluación (Tabla IX) permitirán comparar objetivamente el comportamiento de los ocho modelos y confirmar la hipótesis de que BM25 constituye una base sólida para un sistema RAG de primera fase antes de incorporar técnicas de recuperación densa. La separación *backend/frontend* introducida en la fase 2 deja lista la infraestructura para reutilizar el API desde los experimentos del módulo 2 sobre *chunking* semántico, *embeddings* y bases de datos vectoriales.

REFERENCIAS

- [1] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

- [2] S. Robertson y H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, n.º 4, pp. 333–389, 2009.
- [3] D. Dohan, “rank-bm25: A Python implementation of the BM25 ranking algorithm,” *GitHub repository*, 2021. [En línea]. Disponible en: https://github.com/dorianbrown/rank_bm25
- [4] Ollama, “Run large language models locally,” 2024. [En línea]. Disponible en: <https://ollama.com>
- [5] A. Abid, A. Abdalla, A. Abid *et al.*, “Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild,” *arXiv preprint arXiv:1906.02569*, 2019.
- [6] M. Neumann, “markdownify: A Python library to convert HTML to Markdown,” *GitHub repository*, 2020. [En línea]. Disponible en: <https://github.com/matthewwithanm/python-markdownify>
- [7] H. Touvron, L. Martin, K. Stone *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [8] Gemma Team, Google DeepMind, “Gemma: Open Models Based on Gemini Research and Technology,” *arXiv preprint arXiv:2403.08295*, 2024.
- [9] OpenAI, “GPT-4o System Card,” 2024. [En línea]. Disponible en: <https://openai.com/index/gpt-4o-system-card>
- [10] OpenAI, “OpenAI API Documentation,” 2024. [En línea]. Disponible en: <https://platform.openai.com/docs>
- [11] S. Ramirez, “FastAPI: Modern, fast (high-performance), web framework for building APIs with Python,” 2024. [En línea]. Disponible en: <https://fastapi.tiangolo.com/>
- [12] T. Christie, “Uvicorn: The lightning-fast ASGI server,” 2024. [En línea]. Disponible en: <https://www.uvicorn.org/>
- [13] S. Nakamura, “sse-starlette: Server-Sent Events for Starlette and FastAPI,” *GitHub repository*, 2024. [En línea]. Disponible en: <https://github.com/syysid/sse-starlette>
- [14] Meta React Team, “React 19 — The library for web and native user interfaces,” 2024. [En línea]. Disponible en: <https://react.dev/>
- [15] E. You *et al.*, “Vite: Next Generation Frontend Tooling,” 2024. [En línea]. Disponible en: <https://vitejs.dev/>
- [16] Tailwind Labs, “Tailwind CSS v4,” 2024. [En línea]. Disponible en: <https://tailwindcss.com/>
- [17] shadcn, “shadcn/ui: Beautifully designed components built with Radix UI and Tailwind CSS,” *GitHub repository*, 2024. [En línea]. Disponible en: <https://ui.shadcn.com/>
- [18] Vercel, “Vercel AI SDK,” *GitHub repository*, 2024. [En línea]. Disponible en: <https://sdk.vercel.ai/docs>
- [19] F. E. Daza, A. F. López, V. Valle Fernández y Á. J. Barco, “ADR decision-2: Migración del frontend de Gradio a React 19 + Vite 7 y adición de backend FastAPI + SSE,” Proyecto interno UAO, 2026.